

Open AI Cubes

Santa Cruz Basilio, Jesus Diego
Serrano Arostegui, Max Jairo
López Arteaga, César Omar
Pumayauli Evangelista, Bryan Jose

Universidad Nacional de Ingenieria - Facultad de Ciencias.
Ciencias de la Computacion

Mayo 2026

Outline

Desarrollo - Implementacion - Pruebas

Introducción y Conceptos

Arquitectura

Dimensiones de IA

Implementación en JAVA

Implementación en Python

Implementación en C++

Resultados

Conclusiones

Introducción y Conceptos

- ▶ **Concepto:** Arquitectura modular donde cada "Cubo" encapsula una dimensión de IA (texto, imagen, audio).
- ▶ **Micro Chunk:** Unidad pequeña de cómputo ejecutable de forma concurrente o distribuida.
- ▶ **Arquitectura Cliente-Servidor:** El servidor centraliza los modelos pesados; el cliente es un consumidor ligero.
- ▶ **Desarrollo Puro:** Implementación exclusiva con el SDK estándar de Java, sin librerías externas.

Valor Agregado

Evita arquitecturas monolíticas y permite la escalabilidad mediante un protocolo de comunicación común.

Arquitectura General del Sistema

Separación en tres zonas críticas: **Entrenamiento, Servidor y Cliente.**

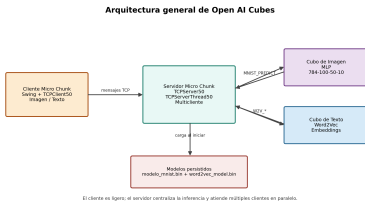
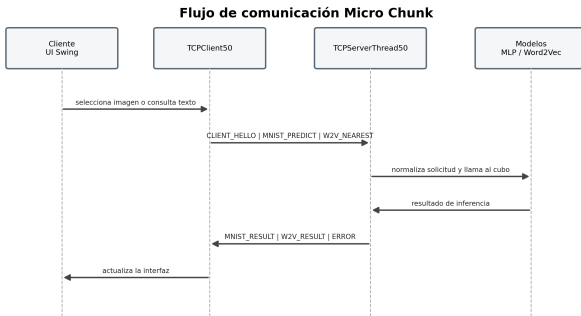


Figure 1: Arquitectura general: Entrenamiento -¿ Servidor -¿ Cliente

- ▶ **Servidor:** Carga binarios y atiende mediante TCPServerThread50.
- ▶ **Protocolo:** Mensajes estructurados por TCP (Módulo — Operación — Payload).

Flujo del Protocolo de Comunicación



Cada cliente conectado posee su propio hilo en el servidor; los modelos son recursos compartidos protegidos durante la inferencia.

Figure 2: Interacción entre cliente, capa TCP y modelos de inferencia

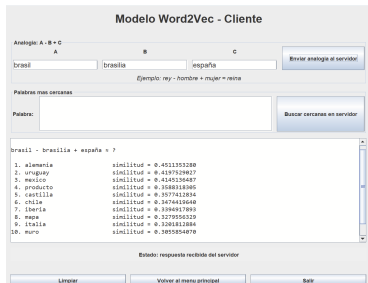
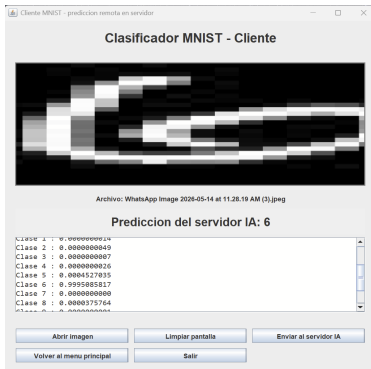
Dimensiones Implementadas

I—X

Cubo Capacidad y Detalle Técnico

Imagen Clasificación MNIST (MLP 784-100-50-10), activación ReLU/Softmax.

Texto Word2Vec Skip-gram, embeddings de 100 dim, analogías y similitud.



Implementación en JAVA: Cliente y Red

- ▶ **TCPClient50:** Gestión de conexión persistente y asíncrona.
- ▶ **Seguridad en Hilos:** Uso de CopyOnWriteArrayList para manejar suscriptores de mensajes.
- ▶ **UI:** Basada en JFrame y Swing, desacoplada de la lógica de red.

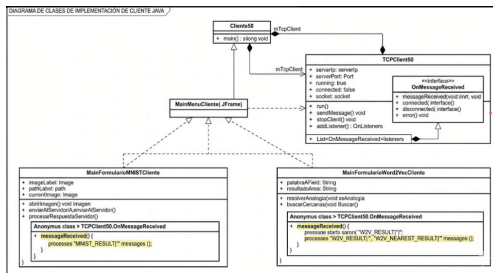


Figure 3: Diagrama de Clases: Arquitectura del Cliente Java

Clase TCPCliente

```
public void run() {
    running = true;
    try {
        // Inicialización de Sockets y flujos de datos...

        String serverMessage;
        // Bucle de escucha persistente
        while (running && (serverMessage = in.readLine()) != null) {
            notifyMessage(serverMessage);
        }
    } catch (Exception e) {
        // Captura de caídas de red o servidor
        notifyError("Error conectando a " + serverIp + ":" + serverPort + " -> " + e.getMessage());
    } finally {
        // Bloque de limpieza garantizada
        running = false;
        connected = false;
        try {
            if (socket != null) {
                socket.close(); // Liberación de recursos
            }
        } catch (Exception ignored) { }
        notifyDisconnected(); // Notifica a la interfaz que se perdió la conexión
    }
}
```

Figure 4: Para manejar desconexiones abruptas

Desacoplamiento y Seguridad

```
// En TCPClient50.java
private final List<OnMessageReceived> listeners = new CopyOnWriteArrayList<>();

private void notifyMessage(String message) {
    for (OnMessageReceived listener : listeners) {
        listener.messageReceived(message); // Notifica a todos los "Cubes" suscritos
    }
}
```

Figure 5: Notifica a los Cubes

```
// En MainFormularioMnistCliente.java
listener = new TCPClient50.OnMessageReceived() {
    @Override
    public void messageReceived(String message) {
        if (message.startsWith("MNIST_RESULT")) {
            // Protección del hilo gráfico
            SwingUtilities.invokeLater(() -> procesarRespuestaServidor(message));
        }
    }
};

// Fragmento de la normalización del vector
entrada[y * 28 + x] = gray / 255.0;
```

Figure 6: Seguridad de la Interfaz

Secuencia de Conexión

1. **Inicio (cliente_main.py):** Pide IP, puerto y nombre del cliente
2. **Conexión (tcp_client_50.py):** Establece socket TCP con servidor
3. **Registro (main_menu_cliente.py):** Envía CLIENT_HELLO|NombreCliente
4. **Menú:** Usuario elige modelo (MNIST o Word2Vec)
5. **Procesamiento:** Envía datos y recibe respuesta del servidor

Uso de librerías para Python

- ▶ **Tkinter:** Librería estándar de Python encargada de la construcción de la interfaz gráfica de usuario (GUI) y la gestión del bucle de eventos principal.
- ▶ **Socket:** Módulo fundamental para la comunicación de red, permitiendo la creación de sockets TCP/IP para el intercambio de datos con el servidor Java.
- ▶ **Threading:** Utilizada para la implementación de la concurrencia, permitiendo que la recepción de datos ocurra en un hilo secundario sin bloquear la interfaz.
- ▶ **Pillow (PIL):** Librería externa empleada para el procesamiento de imágenes en el módulo MNIST, encargada del redimensionado y la normalización de píxeles.
- ▶ **OS:** Librería del sistema utilizada para la gestión de rutas y archivos locales.

tcp_client_50.py - Núcleo de Conexión

Este módulo encapsula la lógica de *sockets* de bajo nivel. Utiliza un hilo secundario. El cliente realiza el pre-procesamiento:

- ▶ Redimensionamiento a 28x28 píxeles.
- ▶ Conversión a escala de grises.
- ▶ Normalización de valores al rango $[0, 1]$.
- ▶ Serialización a formato CSV para su transmisión.

tcp_client_50.py - Núcleo de Conexión

```
1 def send_message(self, message):
2     if self.socket and self.connected:
3         try:
4             # Agrega salto de línea y codifica UTF-8
5             self.socket.sendall((message + "\n")
6                                 .encode('utf-8'))
7             return True
8         except:
9             self.connected = False
10            self.notify_error("Conexión perdida al enviar.")
11            return False
12     else:
13         self.notify_error("No hay conexión activa.")
14         return False
```

Figure 7: send messages

Procesamiento de Imágenes: formmnist.py

A diferencia de implementaciones en otros lenguajes, la versión en Python aprovecha la librería `Pillow` para el tratamiento de datos. El cliente realiza el pre-procesamiento:

- ▶ Redimensionamiento a 28x28 píxeles.
- ▶ Conversión a escala de grises.
- ▶ Normalización de valores al rango $[0, 1]$.
- ▶ Serialización a formato CSV para su transmisión.

Procesamiento de Lenguaje Natural (form_w2v.py)

Este módulo implementa la interfaz para el modelo Word2Vec. Se encarga de capturar las analogías (A es a B como C es a X) y enviarlas mediante el comando `W2V_ANALOGY`

Procesamiento de Lenguaje Natural (form_w2v.py)

```
1 def buscar_cercanas(self):  
2     palabra = self.palabra_nearest.get().strip()  
3  
4     if not palabra:  
5         messagebox.showwarning("Aviso",  
6             "Ingresa una palabra.")  
7         return  
8  
9     # Busca las 10 palabras m s similares  
10    self.tcp_client.send_message(  
11        f"W2V_NEAREST|{palabra}|10")
```

Figure 8: buscar cercanas

Implementación en C++: Qt Framework

- ▶ **Herramientas:** Qt Designer (.ui) y QTcpSocket.
- ▶ **Compatibilidad:** Demuestra la capacidad del servidor Java para atender clientes heterogéneos.

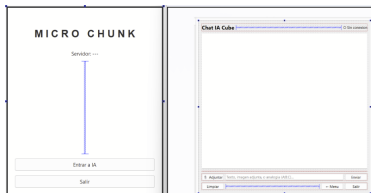


Figure 9: Interfaces .ui en Qt

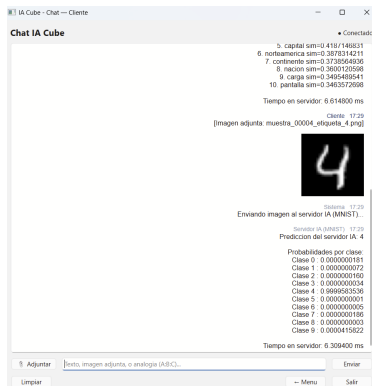


Figure 10: Chat de mensajería C++

Diagrama de Secuencia C++

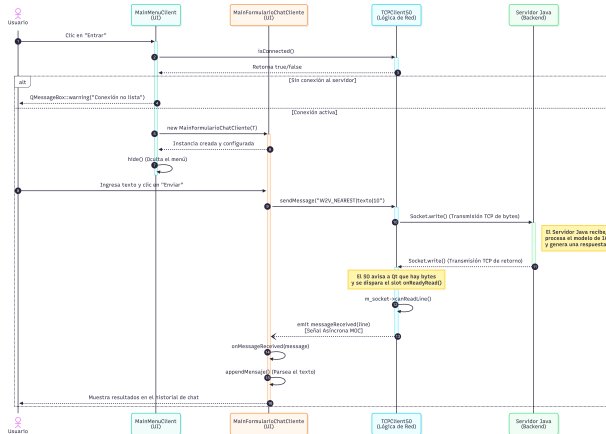
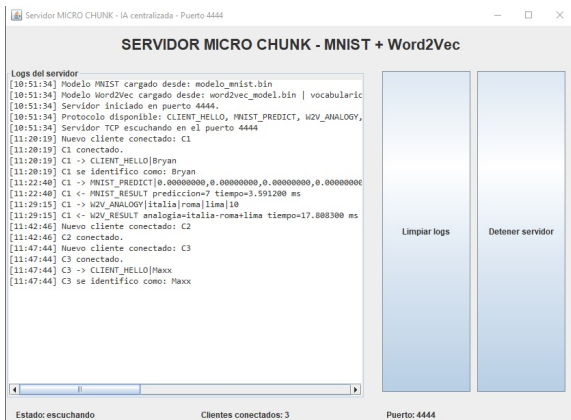


Figure 11: Flujo de mensajes: QTcpSocket -¿ Java TCPServerThread

Resultados: Pruebas en Paralelo

- ▶ **Entorno:** Windows 10, i7-7700 (8 hilos), 16GB RAM.
- ▶ **Prueba:** 3 clientes simultáneos (Java, Python, C++) conectados al servidor Java.



```
Logs del servidor
[10:51:34] Modelo MNIST cargado desde: modelo_mnist.bin
[10:51:34] Modelo Word2Vec cargado desde: word2vec_model.bin | vocabulario
[10:51:34] Servidor iniciado en puerto 4444.
[10:51:34] Protocolo disponible: CLIENT_HELLO, MNIST_PREDICT, W2V_ANALOGY,
[10:51:34] Servidor TCP escuchando en el puerto 4444
[11:20:19] Nuevo cliente conectado: C1
[11:20:19] C1 conectado.
[11:20:19] C1 -> CLIENT_HELLO|Bryan
[11:20:19] C1 se identifica como: Bryan
[11:22:40] C1 -> MNIST_PREDICT|0.00000000,0.00000000,0.00000000,0.00000000
[11:22:40] C1 -> MNIST_RESULT prediccion=7 tiempo=3.591200 ms
[11:29:15] C1 -> W2V_ANALOGY|italia|roma|lima|10
[11:29:15] C1 -> W2V_RESULT analogia=italia-roma+lima tiempo=17.808300 ms
[11:42:46] Nuevo cliente conectado: C2
[11:42:46] C2 conectado.
[11:47:44] Nuevo cliente conectado: C3
[11:47:44] C3 conectado.
[11:47:44] C3 -> CLIENT_HELLO|Maxx
[11:47:44] C3 se identifica como: Maxx
```

Estado: escuchando Clientes conectados: 3 Puerto: 4444

Figure 12: Servidor atendiendo múltiples hilos/lenguajes en paralelo

Resultados: Sistemas Distribuidos

- **Hardware:** 4x Raspberry Pi 4B (Debian 13).
- **Configuración:** 1 RPi como Servidor, 3 RPi como Clientes.

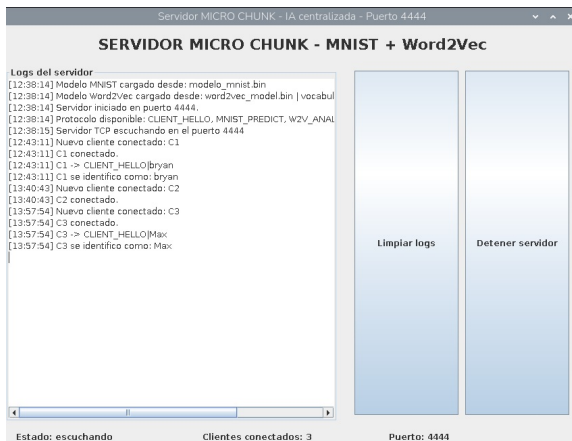


Figure 13: Despliegue distribuido real en nodos Raspberry Pi

Conclusiones

- ▶ **Funcionalidad:** Se materializó Open AI Cubes para texto e imagen con un 97.06% de precisión en MNIST.
- ▶ **Concurrencia:** El esquema de un hilo por cliente permite escalabilidad sin bloqueos en la inferencia.
- ▶ **Interoperabilidad:** El protocolo basado en TCP permite la convivencia de Java, C++ y Python de forma transparente.
- ▶ **Arquitectura:** El diseño es extensible a nuevas dimensiones (Audio/Sensores) mediante la adición de nuevos Cubos.

¡Gracias!